Image Classification with a Backpropagation NN

1 Dataset Origin

The CIFAR-10 was found on the provided website: https://www.imageprocessingplace.com/root_files_V3/image_databases.htm. This dataset is also available in the Tensorflow datasets, so that was the source utilized. The dataset's homepage is https://www.cs.toronto.edu/~kriz/cifar.html

It has 10 objects or animals that act as categories for each of the 60000 images in the dataset. The dataset is balanced, offering a similar number of pictures of each of the categories.

1.1 Technical Description

The TensorFlow library was used to create a backpropagation Neural Network to classify 32x32-pixel color images into their categories. These networks learn by first evaluating one (online) or more (batch) input samples and checking with the input's label. The comparison with the input's label is used to evaluate whether the model is close to understanding the images and why it should classify each in one place or another. From the error calculated (loss), the model readjusts its weights using a gradient descent function.

1.2 Algorithm Design

1.2.1 Data Preparation

The dataset was loaded from TensorFlow, already split into train and test splits. This data is already prepared, containing 32x32 pixels with RGB values. The shape of the image vectors is (60000, 32, 32, 3). 60000 images total, 32x32 pixels, 3 values per pixel. The image data values are 0-255, meaning a 8-bit per color configuration. For the NN to learn better, this is normalized to a value between 0 and 1.

1.2.2 Model Architecture

The model built is a Convolutional NN. This is a Neural Network with convolutional and pooling layers. The global architecture is:

1. Input
2. Convolutional and MaxPool Layers: Conv1 -> MaxPool1 -> Conv2 -> MaxPool2-> Conv3
3. Flatten the resulting vector -> Dense1 -> Dense2 (output)

Each of the 3 convolutional layers look at the image in kernels of 3x3. This allows to find patterns between the neighbors of each pixel, allowing to learn about the organization of the pixels rather than only the color of each individual pixel. Having 3 allows the later ones to find more abstract patterns than the first one.

The Maxpool layers are good at giving the model the capacity to find spatial features even if they moved a bit compared to another image, by taking the maximum value on each 2x2 box. Additionally, they reduce the dimensionality to keep computation manageable, while retaining much of the information gained through the convolutional layers.

Finally, to reshape the spatial dimensions that the previous layers create into a 1D vector that can be used to classify, we flatten the final vector and pass this vector to two final Dense layers.

The first layer reduces the dimensions of the vector to a vector of 64 values, which is passed onto the 10 final probabilities. This has been found to work better than one single dense layers, because it forces the network to learn from a more compact dimension set rather than a large number of dimensions before classifying into the final 10 categories.

All the layers are activated with a ReLU activation function. This activation function is fast, and makes gradients easy to calculate in backpropagation networks. It tends to work well for a wide range of varieties of data.

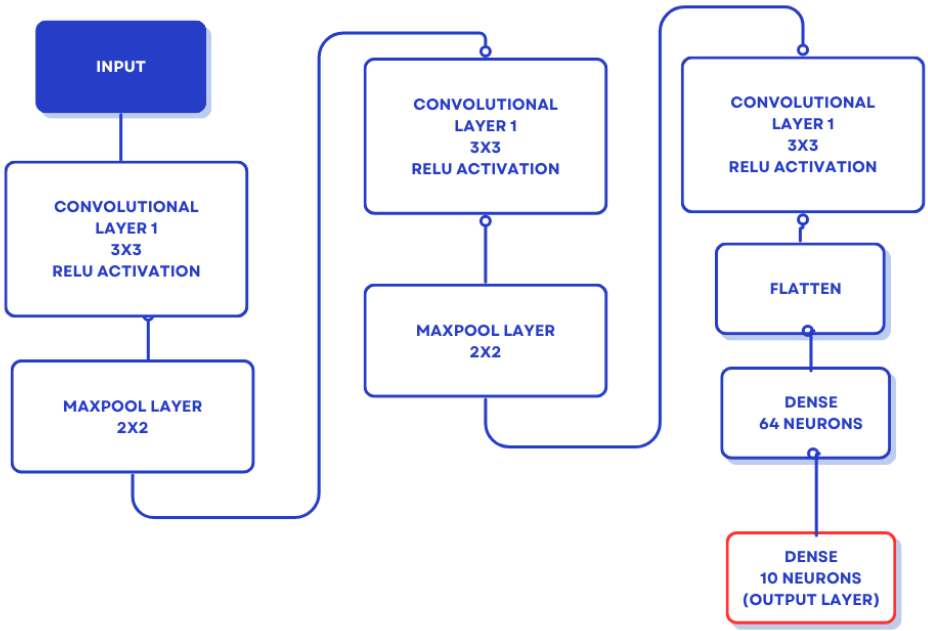
1.2.3 Configurations

The loss metric used is Categorical Crossentropy. This is a common metric used for multi-class classification, which penalizes the confident wrong answers heavily. This is positive because it allows the model to correct these large errors.

The initial weights are selected at random. Starting multiple times gives very similar results, showing that there is no advantage in setting specific ones.

The learning rate is set to 0.0005 and the stopping criterion is stopping after 10 epochs with no fall in validation loss, and reverting to the best model after finishing the 10 extra epochs.

The following flowchart visually displays the layers of the convolutional NN for a structured and clear view:



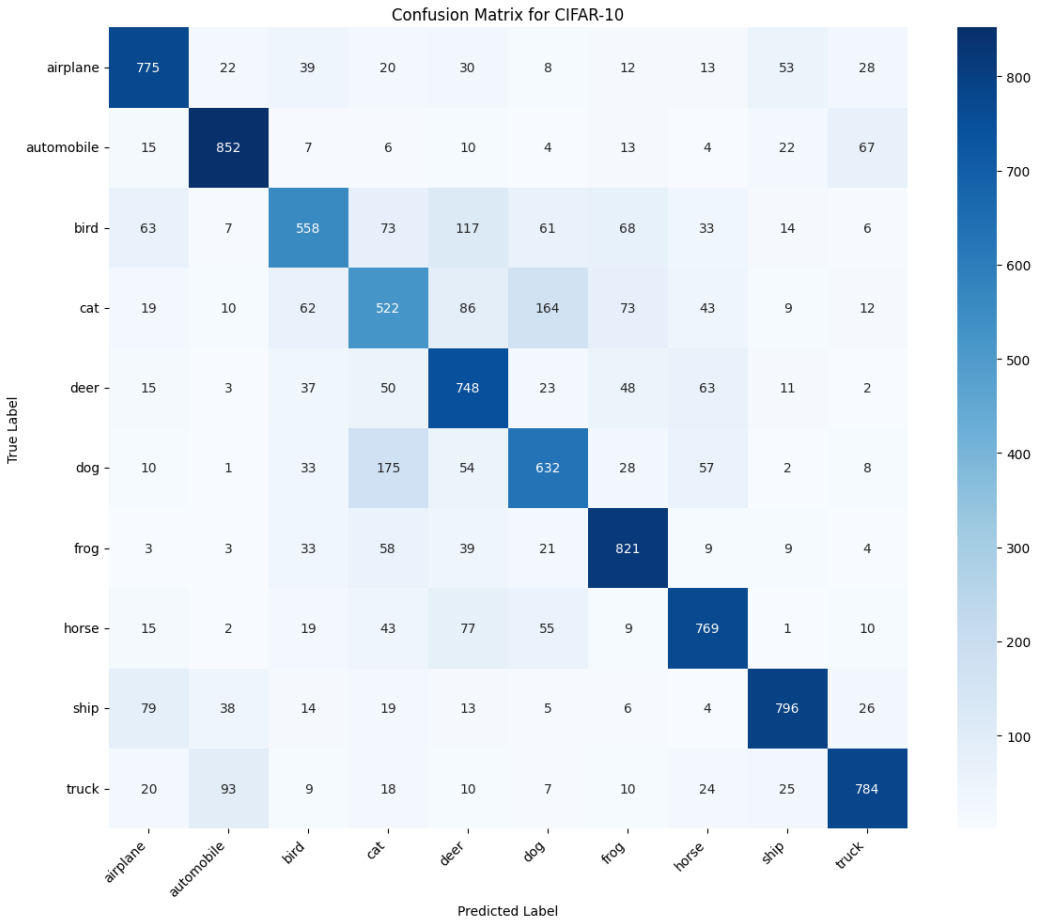
1.3 Results of the Algorithms and Analysis of the Results

The Neural Network achieved a performance that showed it clearly learnt from the images. The best accuracy sat around 0.725, meaning 70% of the images were correctly classified. It was around what was expected, given that the images are 32x32 pixels, meaning some of them can be challenging to classify even for a human. However, it can surely do better because a human

can recognize the images with better accuracy than the model. It is important to note that humans have seen this objects many more times and in many more situations that the neural network, meaning the model could also improve from more images of the 10 categories used.

The accuracy mentioned was calculated after training for 12 epochs, which was determined to be the best after using the mentioned stopping criterion.

Below is the confusion matrix for the described model:



From the confusion matrix, we can see how the model classifies most of the images correctly, particularly struggling in the cat, bird and dog categories. The vehicles tend to be misclassified as other vehicles, and the animals tend to be misclassified as other animals. That is very logical, considering these two groups are very different from each other but similar within the groups.

The comparisons can be made clearly because there is 1000 images of each category in the test set, facilitating the visualization of common confusions, such as cat vs dog.

1.4 CODE:

```
# Load Data
import tensorflow as tf
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.cifar10.load_data()

# Define model
model = tf.keras.models.Sequential([
    tf.keras.layers.Conv2D(32, (3, 3), activation='relu', input_shape=(32, 32, 3)),
    tf.keras.layers.MaxPooling2D((2, 2)),
    tf.keras.layers.Conv2D(64, (3, 3), activation='relu'),
    tf.keras.layers.MaxPooling2D((2, 2)),
    tf.keras.layers.Conv2D(64, (3, 3), activation='relu'),
```

```
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(64, activation='relu'),
    tf.keras.layers.Dense(10)
])
# Normalize pixel data
x_train_normalized = x_train / 255.0
x_test_normalized = x_test / 255.0

# compile the model with loss and optimizers
model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.0005),
    loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True),
    metrics=['accuracy']
)
# Add a stopping criteria
callbacks = [tf.keras.callbacks.EarlyStopping(monitor='val_loss', patience=25,
restore_best_weights=True)]

# Train the model
history = model.fit(x_train_normalized, y_train, epochs=100,
                    validation_data=(x_test_normalized, y_test),
                    callbacks=callbacks)
# TEST MODEL

# Accuracy
test_loss, test_acc = model.evaluate(x_test_normalized, y_test, verbose=2)
# Show
print(f'Test accuracy: {test_acc:.4f}')
print(f'Test loss: {test_loss:.4f}')
# Confusion Matrix
import numpy as np
from sklearn.metrics import confusion_matrix
import matplotlib.pyplot as plt
import seaborn as sns

# Define CIFAR-10 class names
class_names = ['airplane', 'automobile', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse', 'ship',
'truck']

y_pred_logits = model.predict(x_test_normalized)
y_pred_classes = np.argmax(y_pred_logits, axis=1)

conf_matrix = confusion_matrix(y_test, y_pred_classes)

# Plot confusion mtrix
plt.figure(figsize=(12, 10))
sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='Blues',
            xticklabels=class_names, yticklabels=class_names)
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.title('Confusion Matrix for CIFAR-10')
plt.xticks(rotation=45, ha='right')
plt.yticks(rotation=0)
plt.tight_layout()
plt.show()
```